

1 Introduction

Hotels face a significant challenge in managing room reservations due to flexible cancellation policies. Many hotels allow guests to cancel their bookings up to 48 hours before their stay, which often results in last-minute vacancies that are difficult to fill. In my research, I will analyze hotel booking data to predict whether a reservation will be canceled. Accurate cancellation predictions can help hotels optimize their overbooking strategies, mitigate financial risks, and improve occupancy rates. The primary data mining challenge is being able to identify key factors that influence cancellations. I will explore various classification models to determine the best way to predict cancellations.

2 Data Sources

For this project, I will be using data sourced from Kaggle. The dataset originally comes from the article *Hotel Booking Demand Datasets*. The dataset consists of 119,390 rows and 32 columns.

I needed to perform data cleansing on the dataset before beginning exploratory analysis. I removed categorical variables that had too many categories (eg. country, agent, company) and other variables that didn't make sense in the context of this problem (arrival year, arrival day, reservation status, reservation date). I also had to convert the categorical variables with `as.factor` in R. This is important for the response variable is canceled because this was interpreted as an integer prior to converting it to a factor variable. The dataset was reduced from 32 to 25 columns.

The objective of this study is to predict the `is_canceled` column as the response variable, where:

- 0 indicates a fulfilled booking
- 1 represents a cancellation

Some of the key predictor variables that will be evaluated in the model include:

- **lead_time**: Number of days between booking date and arrival date.
- **stays_in_weekend_nights**: Number of weekend nights in the booking.
- **stays_in_week_nights**: Number of week nights in the booking.
- **is_repeated_guest**: Whether the guest is a repeat customer.
- **previous_cancellations**: Whether the guest has made previous cancellations.
- **deposit_type**: Type of deposit made for the booking.

	lead_time	stays_in_weekend_nights	stays_in_week_nights	is_repeated_guest	previous_cancellations	deposit_type	is_canceled
	<int>	<int>	<int>	<int>	<int>	<chr>	<int>
1	342	0	0	0	0	No Deposit	0
2	737	0	0	0	0	No Deposit	0
3	7	0	1	0	0	No Deposit	0
4	13	0	1	0	0	No Deposit	0
5	14	0	2	0	0	No Deposit	0

Figure 1: Snapshot of hotel booking data. Note: there many more fields

Now lets look at exploratory data analysis for hotel bookings. In Figure 2, I created a histogram of lead times that with two distributions laid on top of each other. The blue color is for reservations not cancelled, the red color is for cancelled reservations, and the dark red is the overlap between the two different cancellation types. In the chart, We can see when the lead time is low, we see a majority of the reservations not getting cancelled. Meanwhile when we get to higher lead times, it looks like the number of cancellations is higher than those that are not cancelled.

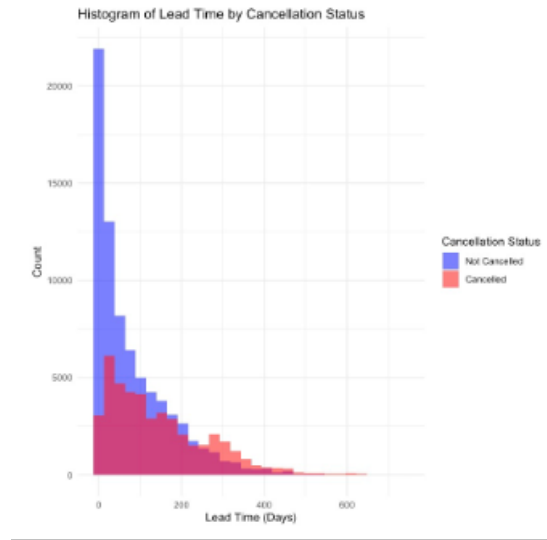


Figure 2: Lead Time Histograms for Cancellation Types

Additionally, I was interested at viewing the number of bookings based on deposit type. Most of the data comes from data with no deposit. However an interesting observation I saw was that non refundable bookings had significantly more cancellations than non cancelled bookings(Figure 3). I would have expected many more cancellations in this deposit type. I believe both cancellation type and lead time will be important variables when building the model.

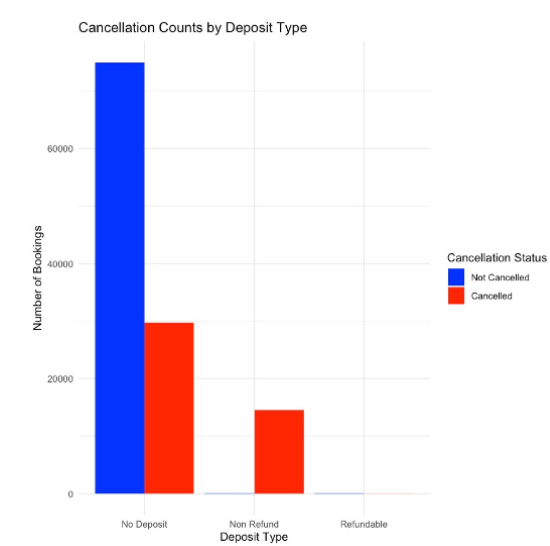


Figure 3: Bar Chart for Number of Bookings based on Depository Type and Cancellation Status

3 Methodology

I started off implementing a logistic regression to use as the baseline model. Logistic regression is a commonly used model for classification problems that estimates probability of an event occurring based on predictor variables. By fitting the model, we found variables that were significant in predicting whether or not a hotel booking would be cancelled. However, due to the complexity of the data, I wanted to implement methods that could capture non-linear relationships in the data.

I used two ensemble methods for capturing model complexity: Random Forest and XGBoost. Random Forest is an ensemble of decision trees where each tree is trained on a random subset of data and features. The predicted value is determined by majority voting from all the trees in the forest. Random Forest builds trees independently and averages the predictions across multiple trees. In contrast, the XGBoost algorithm builds trees sequentially, where each tree is built to correct errors from the previous trees. This iterative process minimizes bias and improves prediction accuracy.

In training, we also used 5-fold cross-validation. This cross validation technique partitions the data into 5 subsets where the model is trained on four and validated on one. This occurs 5 times where each subset serves as the validation set once. I applied cross validation to tune the parameters in both models. In Random Forest, I tuned the hyperparameter *mtry*, which controls the number of features that are evaluated at each split. In XGBoost, I tuned the hyperparameters learning rate, tree depth, and number of boosting rounds. Once the best hyperparameters were found for the trained models, I used those models to classify whether a hotel booking would get cancelled. Finally I used test error to compute the accuracy of the models by measuring the proportion of misclassified instances

4 Results

Model	Test Error
Logistic Regression	0.1887
Random Forest (<i>mtry</i> = 4)	0.1392
XGBoost	0.1698

Table 1: Comparison of Test Error Rates for Different Models

To compare the models performance, I used test error rates as a measure. Test error represents a proportion of incorrect predictions in the test data. Logistic regression was the first model that was tested and it had a test error of .1887. This value suggests the model could capture some relationship in the data but its predictive power is limited. Thus, I wanted to test non-linear models to see if this would improve the test error. The first ensemble method I tested was a random forest model. After performing cross validation for the

mtry hyperparameter, mtry=4 had the lowest testing error at .1392, which was an improvement from logistic regression. By aggregating many decision trees, the Random Forest improved the classification accuracy. Lastly, XGBoost yielded a test error of .1698. While this was an improvement from the baseline model, it did not surpass the random forest. While boosting can be effective, I believe its performance was affected by the hyperparameter tuning and structure of the data. I reduced the range of values for the hyperparameter to evaluate to increase the speed of running the cross-validation.

5 Conclusion

My analysis aimed to predict hotel booking cancellations where I tested logistic regression, Random Forest, and XGBoost. Random Forest achieved a test error of .1392, outperforming both logistic regression and XGBoost. I was surprised boosting did not provide a significant improvement over the logistic regression model since XGBoost is known for high predictive accuracy. This suggests the datasets structure and feature interactions were not well suited for XGBoost. In the future, I can try AdaBoosting to see if this would yield better results. In addition, analyzing feature importance will provide a more interpretable model.

6 Bibliography

1. <https://www.kaggle.com/datasets/jessemostipak/hotel-booking-demand?resource=download>